Technical University of Crete
School of Electrical and Computer Engineering
Course: Randomized Algorithms
Exercise 2 (100/1000)
Report Delivery Date: 19/11/2024
Instructor: Athanasios P. Liavas

Paraskevopoulou Despoina
2021030135

Maximum Load in n-Balls and n-Bins experiment

In this first exercise, we are simulating the Balls and Bins problem. More specifically, we are throwing n balls into n bins, independently and uniformly at random. For that simulation, we computed the maximum load. We repeated the experiment K times and we plotted the theoretical high porbability upper bound ($M = 3\frac{lnn}{lnlnn}$) and lower bound ($L = \frac{lnn}{lnlnn}$).

Below, we represent the results for various n and K=100.

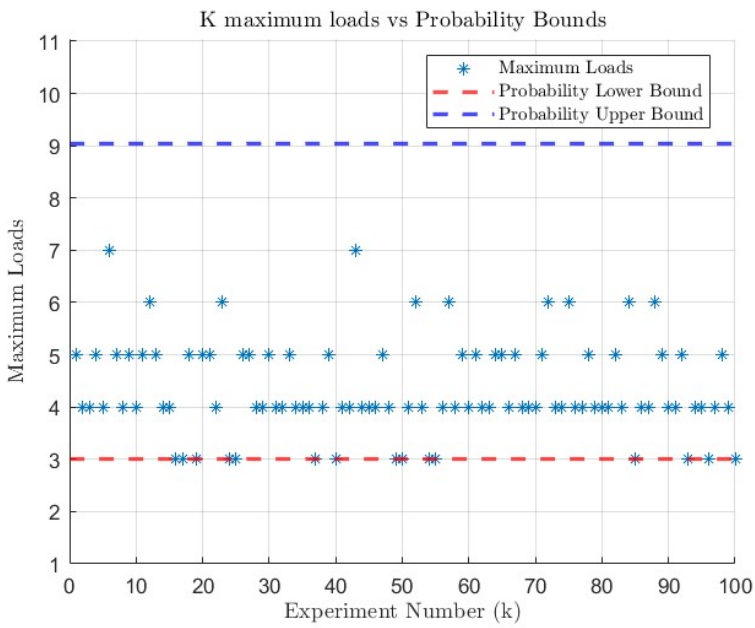

Figure 1: Maximum loads vs Probability Bounds, n=100

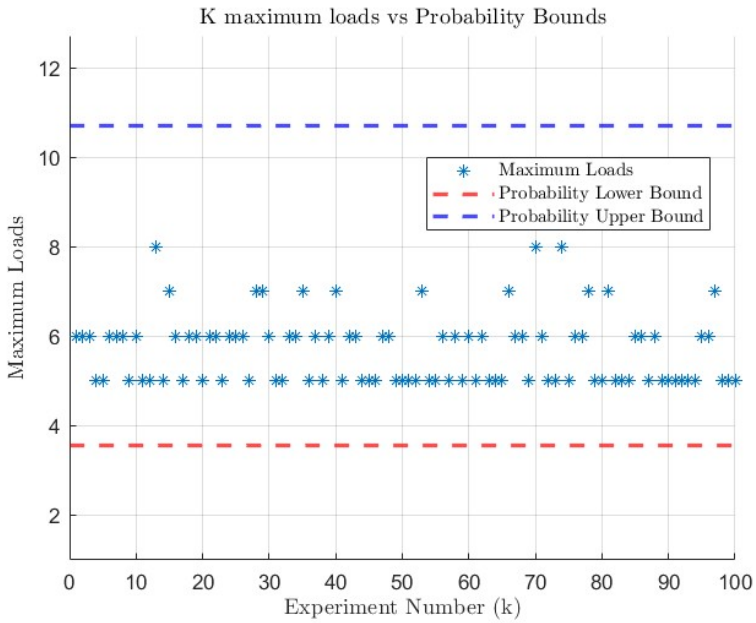

Figure 2: Maximum loads vs Probability Bounds, n=1000

The code for the implementation is:

```
function max_loads = balls_bins_rep(n,K)
    B = zeros(1,K);
    max_loads = zeros(1,K);
    for i = 1:K
        bins = zeros(1,n);
        balls = zeros(1,n);
        ball_i = 0;

        while ball_i < n
            ball = randi(n,1,n);

            for b = ball
                bins(b) = bins(b)+1;
                ball_i = ball_i + 1;
            end
        end
        max_loads(i) = max(bins);
    end
end
```

Observations: We observe that the maximum balls in a bin are always below the theoretical upper bound. The minimum balls in a bin, for small n (n=100), are equal to the theoretical lower bound, but for larger n they are always above the bound.

Hamiltonian Cycles

For the second part of this assignment, we studied and implemented the modified algorithm for Hamiltonian cycles.

The algorithm we followed for the implementation of the problem of Hamiltonian Cycles is given below.

Algorithm 1 Hamilton Cycles Implementation

```
q ← 20 logn/n
p ← 2q − q^2
Create random graph G_{n,p}
for every node v do
    Create unusedEdges(v)
end for
while true do
    Run Modified Algorithm for Hamilton Cycles for PATH
    if iterations > 2nlogn then
        FAIL
    end if
end while
while true do
    Run Modified Algorithm for Hamilton Cycles for CYCLE
    if iterations > nlogn then
        FAIL
    end if
end while
```

In our implementation, both algorithm stages were successful. We returned the iterations that were needed for each part. For $n = 500$, the iterations for the path are in the range of 2500 to 4000. Some indicative results are 3654, 2688, 3705, 3427. For the cycle the range is from 30 to 2000, with numbers as 454. 394, 652, 2014 respectively.

In order to verify the correctness of our algorithm, we created a function which checks the following cases:

1. We check the length of the path, which should be $n + 1$, if what we found is a cycle.
2. We check whether the end and the beginning of the path are the same. If it is a cycle, they have to be the same.
3. We check whether the nodes are unique.
4. Last check is whether the nodes are adjacent to each other.

The code for the function described above, is given below:

```
function is_valid = verify_hamiltonian_cycle(path,n,adjMatrix)

    if length(path) ~= n+1
        disp('Cycle verification failed: Incorrect cycle length.');
        is_valid = false;
        return;
    end

    if path(1) ~= path(end)
        disp('Cycle verification failed: Start and end nodes do not match.');
        is_valid = false;
        return;
    end

    if length(unique(path(1:end-1))) ~= n
        disp('Cycle verification failed: Not all nodes are unique.');
        is_valid = false;
        return;
    end

    for i=length(path)-1
        if(adjMatrix(path(i),path(i+1))==0)
            is_valid = false;
            return;
        end
    end

    is_valid = true;
end
```

Here is the code for the hamiltonian cycle problem.

```
function [count1,count2,success]=hamiltonian_cycle_func(n,q,p)
adjMatrix = zeros(n);

for i =1:n
    for j=(i+1):n
        if rand() < p
            adjMatrix(i,j) = 1;
        elseif rand() >= p
            adjMatrix(i,j) = 0;
        end
    end
end

adjMatrix = adjMatrix + adjMatrix';
usedEdges = cell(1,n);
unusedEdges = cell(1,n);

for i = 1:n
    for j = (i+1) : n
        if(adjMatrix(i,j)==1)
            w = rand();
            if w <  q
                unusedEdges{i} = [unusedEdges{i},j];
            elseif w<p
                unusedEdges{j} = [unusedEdges{j},i];
            else
                unusedEdges{i} = [unusedEdges{i},j];
                unusedEdges{j} = [unusedEdges{j},i];
            end
        end
    end
end

for i=1:n
    unusedEdges{i} = unusedEdges{i}(randperm(length(
       unusedEdges{i})));
end

% random vertex as head
head = randi(n);
path = [head];
circle_found = false;
path_found = false;
count1 = 0;

while ~path_found
    r = rand();
    count1 = count1+1;

    if length(path) == n
        fprintf('Path found, iterations: %d \n',count1);
        break; % Hamiltonian path found
    end

    if isempty(unusedEdges{head})
        fprintf('Path NOT found, iterations: %d \n',count1);
```

```
        path_found = false;
        break;
    end

    if r<1/n
        head = path(1);
        path = flip(path);
    elseif r < 1/n + length(usedEdges{head})/n
        v = usedEdges{head}(randi([1,length(usedEdges{head})]));
        [exists,position] = check_vertex(path,v);

        if exists
            head = path(position+1);
            path = rotate_path(path,position);
        end
    else
        v = unusedEdges{head}(1);
        unusedEdges{head}(1) = [];
        usedEdges{head} = [usedEdges{head},v];

        [exists,position] = check_vertex(path,v);
        if ~exists
            head = v;
            path = [path,v];
        else
            if position == length(path)
                        head = path(1);
                        path = rotate_path(path,position);
                else
                        head = path(position+1);
                        path = rotate_path(path,position);
                    end
        end
    end

    if isempty(unusedEdges)
         error('FAIL: Unused edges exhausted, iterations; %d \n',count1);
    end
    if count1 > 2 * n * log(n)
        fprintf('FAIL: Maximum iteration limit exceeded for path, iterations; %d \n',count1);
        break;
    end

end
count2 = 0;

while circle_found==false
    r = rand;
    count2 = count2+1;

    if isempty(unusedEdges{head})
        fprintf('Cycle NOT found, iterations: %d \n',count2);
        circle_found = false;
        break;
    end
```

```
    if r<1/n
        head = path(1);
        path = flip(path);
    elseif r < 1/n + (length(usedEdges{head})/n)
        v = usedEdges{head}(randi([1,length(usedEdges{head})]));
        [exists,position] = check_vertex(path,v);
        if exists
            head = path(position+1);
            path = rotate_path(path,position);
        end
    else
        v = unusedEdges{head}(1);
        unusedEdges{head}(1) = [];
        usedEdges{head} = [usedEdges{head},v];

        [exists,position] = check_vertex(path,v);
        if v==path(1)
            path = [path,path(1)];
            circle_found = true;
            fprintf('Cycle found, iterations: %d\n',count2);
            break;
        else
            if ~exists
                path = [path,v];
                head = v;
            else
               if position == length(path)
                        head = path(1);
                        path = rotate_path(path,position);
               else
                        head = path(position+1);
                        path = rotate_path(path,position);
               end
            end
        end
    end

    if isempty(unusedEdges)
         error('FAIL: Unused edges exhausted, iterations; %d \n',count2);
    end
    if count2 >  n * log(n)
        fprintf('FAIL: Maximum iteration limit exceeded for path, iterations; %d \n',count2);
        break;
    end

end
is_valid = verify_hamiltonian_cycle(path,n,adjMatrix);
if is_valid
        success = 1;
    else
        success = 0;
end
end
```